

The Many Approaches to Sandboxing in Linux

You can isolate malicious programs or risky tasks by sandboxing them in different ways to stop them from affecting your main system. This article gives the reader a working knowledge of sandboxing in Linux.

Securing your system is a big priority for every production environment, whether you are a systems admin or a software developer. The best way to secure your operating system from doubtful programs or processes is by sandboxing (also termed as jailing). Sandboxing involves providing a safe environment for a program or software

so that you can play around with it without hurting your system. It actually keeps your program isolated from the rest of the system, by using any one of the different methods available in the Linux kernel. Sandboxing can be useful to systems administrators if they want to test their tasks without any damage and also to developers for testing their

pieces of code. A sandbox can help you to create a different environment from your base operating system. It has become trendy due to its extensive use by PaaS and SaaS providers.

The idea of jailing is not new since it has been available in UNIX based BSD OSs. For years, BSD has used the concept of 'jails', while Solaris has used 'zones'. But in Linux, this concept was started with *chroot* and has been possible because *namespaces* are present in the Linux kernel.

Namespaces

Namespaces are features available in Linux to isolate processes in different system resource aspects. There are six types of namespaces available up to kernel 4.0. And more will be added in the future. These are:

- *mnt* (mount points, file systems)
- *pid* (processes)
- *net* (network stack)
- *ipc* (system V IPC)
- *uts* (host name)
- *user* (UIDs)

Linux namespaces are not new. The first one was added to Linux in 2008 (Linux kernel 2.6), but they became more

widely used only in Linux kernel 3.6, when work on the most complex of them all — the *users namespace* — was completed. Linux kernel uses *clone()*, *unshare()* and *setns()* system calls to create and control namespaces.

Creation of new namespaces is done by the *clone()* system call, which is also used to start a process. The *setns()* system

call adds a running process to the existing namespace. The *unshare()* call works on a process inside the namespace, and makes the caller a member of the namespace. Its main purpose is to isolate the namespace without having to create a new process or thread (as is done by *clone()*). You can directly use some services to get the features of these namespaces. *CLONE_NEW** identifiers are used

with these system calls to identify the type of namespace.

These three system calls make use of the *CLONE_NEW** as *CLONE_NEWIPC*, *CLONE_NEWNS*, *CLONE_NEWNET*, *CLONE_NEWPID*, *CLONE_NEWUSER*, and *CLONE_NEWUTS*. A process in a namespace can be different because of its unique inode number when it is created.

```
#ls -al /proc/<pid>/ns
lrwxrwxrwx 1 root root 0 Feb 7 13:52 ipc -> ipc:[4026532253]
lrwxrwxrwx 1 root root 0 Feb 7 15:39 mnt -> mnt:[4026532251]
lrwxrwxrwx 1 root root 0 Feb 7 13:52 net -> net:[4026531957]
lrwxrwxrwx 1 root root 0 Feb 7 13:52 pid -> pid:[4026532254]
lrwxrwxrwx 1 root root 0 Feb 7 13:52 user ->
user:[4026531837]
lrwxrwxrwx 1 root root 0 Feb 7 15:39 uts -> uts:[4026532252]
```

Mount namespace: A process views different mount points other than the original system mount point. It creates a separate file system tree associated with different processes, which restricts them from making changes to the root file system.

PID namespace: PID namespace isolates a process ID from

